

AI Endpoint Intelligence Platform

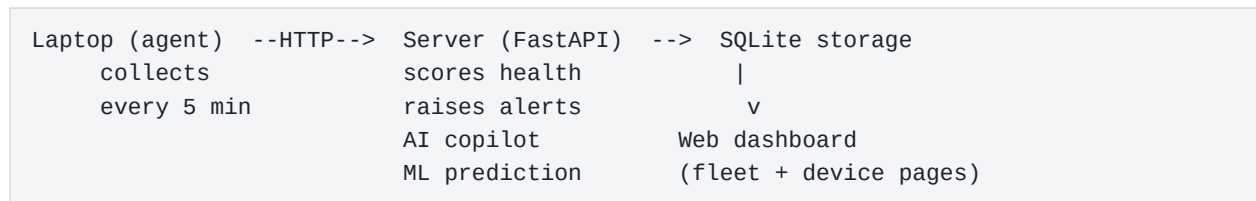
Complete deployment guide: standing up the server, deploying the agent to a laptop fleet, and integrating with an existing provisioning workflow.

Windows 11 laptop fleet • Python agent + FastAPI server • SYSTEM scheduled-task deployment

1. What you are deploying

The platform has two halves. A lightweight **agent** runs on every laptop, collecting health telemetry (CPU, memory, disk, battery, SMART disk health, and hardware identity) and sending it to a central **server**. The server stores the data, scores each device's health, raises alerts (including for machines that go silent), explains issues in plain language via an AI copilot, and optionally predicts drive failure. A web dashboard presents all of it.

Data flow



Project layout

endpoint-intelligence/	
agent/	runs on each laptop (agent.py, collectors.py, config.py ...)
server/	runs on one always-on machine (main.py, health.py ...)
deploy/	the installer kit (install.ps1, run_agent.bat ...)
ml/	optional failure-prediction model (train.py, predict.py)

Deployment in three phases

Do them strictly in order: (1) stand up the server and confirm it is reachable, (2) prove one agent works end to end, (3) roll out to the fleet. Never deploy agents before the server is confirmed reachable, or you will be debugging two moving parts at once.

2. Prerequisites

Server machine

- A machine that stays powered on and is reachable by the laptops (a spare desktop, small server, or VM on the same network).
- Python 3.10 or newer installed.
- A fixed or reserved LAN IP address is strongly recommended so agents always know where to find it.

Each laptop

- Windows 11 (the fleet standard).
- Network route to the server on the chosen port (default 8000).
- Administrator rights available during installation (the installer needs them to register a SYSTEM scheduled task).
- Python and smartmontools — the installer fetches these via winget if missing, or they can be baked into the base image (recommended).

Bake dependencies into your image

Because you already provision machines from a standard image, install Python and smartmontools once in that image. Then per-machine deployment has no internet dependency and is faster and more reliable — the installer only handles the agent files and the scheduled task.

3. Phase 1 — Stand up the server

Do this once, on the always-on machine. Copy the `server/` and `ml/` folders to it.

3.1 Install dependencies and start

```
cd server
pip install fastapi uvicorn jinja2 requests
uvicorn main:app --host 0.0.0.0 --port 8000
```

The `--host 0.0.0.0` flag is mandatory

Without it the server only listens on localhost and no laptop can reach it. `0.0.0.0` makes it accept connections from other machines on the network.

3.2 Find the server's IP address

On the server, run `ipconfig` and note the IPv4 address (for example `192.168.1.50`). Every agent will point at this address.

3.3 Open the firewall port

If other machines cannot reach the server, Windows Firewall is almost always the cause. On the server, in an elevated PowerShell:

```
New-NetFirewallRule -DisplayName "Endpoint Server" `
-Direction Inbound -LocalPort 8000 -Protocol TCP -Action Allow
```

3.4 Confirm it is reachable

From a *different* machine on the network, open `http://192.168.1.50:8000/` in a browser. A JSON health-check response confirms the server is up and reachable. If it times out, recheck the IP and the firewall rule before going further.

Making the server permanent

Running uvicorn in a terminal stops when the window closes or you log off — fine for testing. For always-on operation, run the server itself as a scheduled task or service (the same SYSTEM-task technique the agent uses, or a tool such as NSSM). Get it working in a terminal first, then make it permanent once you trust it.

4. Phase 2 — Deploy one test agent

Prove the whole path on a single laptop before touching the fleet.

4.1 Point the agent at the server

Edit `agent/config.py`:

```
MODE = "server"
SERVER_URL = "http://192.168.1.50:8000" # your server's real LAN IP
COLLECT_INTERVAL_SECONDS = 300        # 5 minutes
```

One config for the whole fleet

This same edited `config.py` ships to every laptop unchanged. The mode and server URL are identical across the fleet, which is what makes rollout a copy-and-run operation.

4.2 Run the installer

Copy the project folder to the laptop. Open an **elevated** PowerShell (Run as administrator), change into the `deploy/` folder, and run:

```
powershell -ExecutionPolicy Bypass -File install.ps1
```

The installer:

- confirms it is running as Administrator,
- copies the agent to `C:\ProgramData\EndpointAgent`,
- installs Python dependencies and `smartmontools` if missing,
- registers a scheduled task that runs the agent as the `SYSTEM` account at startup and every 5 minutes,
- runs one collection immediately so data appears right away.

4.3 Verify end to end

Check	Where	Expected
Agent log	<code>C:\ProgramData\EndpointAgent\agent.log</code>	Agent log is present (sent to server)
Dashboard	<code>http://192.168.1.50:8000/dashboard</code>	The laptop appears as a card
SMART data	Device detail page	A 'Disk Health' component is present

Why the agent runs as **SYSTEM**

SMART disk reads require administrator rights. Running the task under the `SYSTEM` account grants those rights with no UAC prompt and lets the agent run even when no user is logged in. If SMART data is missing, confirm in Task Scheduler that the task's account is `SYSTEM`.

5. Phase 3 — Roll out to the fleet

Once one machine works, the rest are identical because the configuration is the same everywhere. There are two paths.

5.1 Simple path — per-machine installer

- Copy the project folder to each laptop (network share, USB, or your existing software-distribution method).
- Run `install.ps1` from an elevated PowerShell on each.
- Same three steps, same config, every machine.

5.2 Recommended path — fold into provisioning

Because you already push a standard software load (antivirus, Office, browser, and so on) during imaging, treat the agent as one more payload. Your provisioning tool's application step can drop the `EndpointAgent` folder and run the task registration exactly the way it runs your other installers. Every machine you image then arrives with health monitoring already active, with zero extra manual steps.

This is the minimal-manual-intervention outcome

Integrating the agent into the existing provisioning flow means new laptops self-enroll into fleet monitoring the moment they are imaged — no separate deployment pass required.

5.3 Decide before scaling wide

- **Base image:** bake in Python and smartmontools to remove the internet dependency at deploy time.
- **Collection interval:** 5 minutes is the default; 10–15 minutes is plenty for health trends and lighter on a large fleet. Set it in both `config.py` and the repetition interval in `install.ps1`.
- **Stale window:** keep `STALE_MINUTES` in `alerts.py` at a few times the collection interval so a single missed cycle does not raise a false alarm.

5.4 Removing the agent

From an elevated PowerShell in `deploy/`:

```
powershell -ExecutionPolicy Bypass -File uninstall.ps1
```

This removes the scheduled task and the installed files. The installer is also idempotent — re-running it cleanly replaces an existing installation, so pushing updates to already-deployed machines is safe.

6. Optional layers

6.1 AI copilot (plain-language explanations)

The copilot turns telemetry into human-readable explanations. Choose where the model runs by setting environment variables on the server before starting it.

Provider	Setup	Trade-off
Ollama (local)	Install Ollama, run 'ollama pull llama3.1'. Default prompter.	Telemetry stays on your network; no per-call cost. Best for privacy.
Anthropic / OpenAI	Set COPILOT_PROVIDER, COPILOT_API_KEY, COPILOT_MODEL; telemetry leaves the network and can be billed.	Costs more; leaves network.

Never hard-code API keys

Keys are read only from environment variables, never written into source files. A key committed to source ends up in backups and shared machines.

6.2 Failure prediction (ML)

Failure prediction is time-gated: a model needs real failures to learn from, and your own fleet will not have enough for months. Start with public data, switch to your own later. The platform runs fine with no model — prediction is an enhancement over the always-on rules.

```
cd ml
pip install scikit-learn pandas joblib numpy
python train.py --demo # synthetic data, proves it works
python train.py --backblaze stats.csv # real public failure data
python train.py --csv fleet_export.csv # your own data, months later
```

Once a model file exists, the server's prediction endpoint uses it automatically; before that it reports 'no model yet'.

7. Configuration reference

The settings you will actually change.

Setting	File	Purpose
MODE	agent/config.py	'local' (testing) or 'server' (deployment)
SERVER_URL	agent/config.py	Address agents send telemetry to
COLLECT_INTERVAL_SECONDS	agent/config.py	How often the agent collects
THRESHOLDS	server/health.py	Warning/critical levels for disk, RAM, CPU
SMART_RULES	server/health.py	SSD wear and sector-count thresholds
STALE_MINUTES	server/alerts.py	When a silent device is flagged 'not reporting'
COPILOT_PROVIDER	environment	ollama, anthropic, or openai
SMARTCTL_PATH	agent/collectors.py	Full path to smartctl if not on PATH

8. Troubleshooting

Symptom	Likely cause and fix
Browser on another machine cannot reach the server	Server not started with --host 0.0.0.0, or Windows Firewall is blocking port 8000. Add the rule.
Agent log shows 'could not reach server'	Wrong SERVER_URL in config.py, server down, or firewall. Verify the IP and that the server is running.
Device appears but has no Disk Health / SMART data	SMART tools not installed, or the task is not running as SYSTEM. Confirm both; test with a known good device.
A machine shows as 'not reporting'	The agent stopped, the machine is off/asleep, or lost network. If expected (e.g. asleep), no action needed.
Copilot button returns an error message	No API key set (cloud) or Ollama not running (local). The rest of the tool is unaffected — see the AI section.
Installer fails: Python not found	winget could not install Python, or no internet. Install Python manually or bake it into the installer.

Your first debugging step is always agent.log

On any misbehaving machine, open C:\ProgramData\EndpointAgent\agent . log and read the last line. It tells you whether the last run collected and sent successfully, or exactly what failed.

9. Go-live checklist

#	Item
1	Server responds at http://<server-ip>:8000/ from another machine
2	Firewall inbound rule for port 8000 is in place
3	config.py has MODE = 'server' and the correct SERVER_URL
4	One test laptop installed; its agent.log shows '(sent to server)'
5	Test laptop appears on the dashboard with a health score
6	SMART / Disk Health data is present on the device detail page

7	Collection interval and STALE_MINUTES tuned for the fleet size
8	Python + smartmontools baked into the base image (recommended)
9	Agent integrated into the provisioning workflow (recommended)
10	Plan in place to run the server permanently (service/task)

Deploy agents early: telemetry accumulates on wall-clock time, so the sooner real machines report in, the sooner you have the history needed for trend analysis and, eventually, fleet-specific failure prediction.